

Atividade – Manipulação de listas em Python

Aluno: Gecinaldo Júnio Vieira Coelho

RGM: 043499317

Turma: D2/Engenharia de Software

Data: 25/08/2026

Código Python

```
# Atividade: identidade e mutabilidade de listas
lista = [10, 20, 30]

def mostrar_estado(etapa):
    print("\n\n" + "=" * 70)
    print(etapa)
    print("Conteúdo atual:", lista)
    print("id() da lista:", id(lista))
    print("Elementos existentes:")
    if len(lista) == 0:
        print(" A lista está vazia.")
    else:
        for indice, valor in enumerate(lista):
            print(f" índice={indice}, valor={valor}, id()={id(valor)}")

mostrar_estado("Lista inicial")

# 1. Adicionar um elemento ao final
lista.append(40)
mostrar_estado("Depois de append(40)")

# 2. Inserir um elemento em uma posição específica
lista.insert(1, 15)
mostrar_estado("Depois de insert(1, 15)")

# 3. Remover um elemento pelo valor
lista.remove(30)
mostrar_estado("Depois de remove(30)")

# 4. Remover um elemento pela posição
valor_removido = lista.pop(2)
print("Valor removido por pop(2):", valor_removido)
```

```

print("id() do valor removido:", id(valor_removido))
mostrar_estado("Depois de pop(2)")
# 5. Alterar o valor de um elemento existente
id_antes_da_alteracao = id(lista[0])
lista[0] = 99
id_depois_da_alteracao = id(lista[0])
print("\nid() do elemento na posição 0 antes da alteração:", id_antes_da_alteracao)
print("id() do elemento na posição 0 depois da alteração:", id_depois_da_alteracao)
mostrar_estado("Depois de alterar lista[0] para 99")
# 6. Limpar a lista
lista.clear()
mostrar_estado("Depois de clear()")

```

Evidências da execução

```

\n=====
Lista inicial
Conteúdo atual: [10, 20, 30]
id() da lista: 138016746115968
Elementos existentes:
    índice=0, valor=10, id()=11278216
    índice=1, valor=20, id()=11278536
    índice=2, valor=30, id()=11278856
\n=====
Depois de append(40)
Conteúdo atual: [10, 20, 30, 40]
id() da lista: 138016746115968
Elementos existentes:
    índice=0, valor=10, id()=11278216
    índice=1, valor=20, id()=11278536
    índice=2, valor=30, id()=11278856
    índice=3, valor=40, id()=11279176

```

```

\n=====
Depois de insert(1, 15)
Conteúdo atual: [10, 15, 20, 30, 40]
id() da lista: 138016746115968
Elementos existentes:
    índice=0, valor=10, id()=11278216
    índice=1, valor=15, id()=11278376
    índice=2, valor=20, id()=11278536
    índice=3, valor=30, id()=11278856
    índice=4, valor=40, id()=11279176
\n=====
Depois de remove(30)
Conteúdo atual: [10, 15, 20, 40]
id() da lista: 138016746115968
Elementos existentes:
    índice=0, valor=10, id()=11278216
    índice=1, valor=15, id()=11278376
    índice=2, valor=20, id()=11278536
    índice=3, valor=40, id()=11279176
Valor removido por pop(2): 20
id() do valor removido: 11278536

Depois de pop(2)
Conteúdo atual: [10, 15, 40]
id() da lista: 138016746115968
Elementos existentes:
    índice=0, valor=10, id()=11278216
    índice=1, valor=15, id()=11278376
    índice=2, valor=40, id()=11279176
\nid() do elemento na posição 0 antes da alteração: 11278216
id() do elemento na posição 0 depois da alteração: 11281064
\n=====
Depois de alterar lista[0] para 99
Conteúdo atual: [99, 15, 40]
id() da lista: 138016746115968
Elementos existentes:
    índice=0, valor=99, id()=11281064
    índice=1, valor=15, id()=11278376
    índice=2, valor=40, id()=11279176
\n=====
Depois de clear()
Conteúdo atual: []
id() da lista: 138016746115968
Elementos existentes:
    A lista está vazia.

```

Respostas da análise

a) O `id()` da lista mudou durante as operações? O que isso indica?

Não. O `id()` da lista permanece o mesmo após `append()`, `insert()`, `remove()`, `pop()`, alteração de elemento e `clear()`. Isso indica que todas essas operações modificam o conteúdo do mesmo objeto lista, sem criar uma nova lista. O `id()` identifica o objeto e se mantém constante durante sua existência.

b) O que aconteceu com os elementos quando novos valores foram adicionados?

Com `append(40)`, o valor foi adicionado ao final da lista. Com `insert(1, 15)`, o valor ocupou o índice 1, deslocando os demais elementos para a direita. Os objetos já existentes continuaram sendo referenciados pela lista, mudando apenas seus índices.

c) O que acontece com a referência de um elemento quando ele é removido da lista?

A referência do elemento removido deixa de pertencer à lista. No `pop()`, o elemento é removido e retornado, podendo ser armazenado em outra variável (como `valor_removido`), mantendo assim uma referência a ele. Se não houver outras referências, o objeto poderá ser liberado pelo gerenciador de memória. O `remove()` elimina a primeira ocorrência de um valor; o `pop()` remove por posição e retorna o elemento.

d) Ao alterar um elemento da lista, o `id()` desse elemento permaneceu igual? Explique o resultado observado.

Não necessariamente. Inicialmente, `lista[0]` referencia o inteiro `10`. Após `lista[0] = 99`, essa posição passa a referenciar o inteiro `99`. Como inteiros são imutáveis, o objeto `10` não é alterado; apenas a referência naquela posição é substituída, o que pode mudar o `id()` observado para esse elemento. O `id()` da lista, porém, continua igual.

e) Por que dizemos que uma lista em Python é um objeto mutável?

Porque seu conteúdo pode ser alterado após a criação: é possível adicionar, remover, substituir e limpar elementos sem criar outro objeto lista. Essas modificações ocorrem no próprio objeto, mantendo o mesmo `id()` da lista.

f) Qual é a diferença entre alterar o conteúdo de uma lista e criar uma nova lista?

Alterar o conteúdo modifica o mesmo objeto, como em:

python

```
lista.append(40)
```

```
lista[0] = 99
```

```
lista.clear()
```

Nesses casos, o `id(lista)` permanece igual. Criar uma nova lista gera outro objeto, como em:

python

```
nova_lista = lista + [40]
```

Aqui, `nova_lista` tem outro `id()`, pois é uma nova lista; a original não é modificada pela concatenação.